

LPP 3CFU – Domande d'esame A.A. 25/26
Parte 2

Luca Roversi

December 5, 2025

CT0300 Functors

Exercise 1. Let $A = \{x, y\}$ and $B = \{p, q\}$ be sets. Let $f = \{(x, p), (y, q)\}$ be a function on sets.

- Draw the diagram resulting from the application of the “Powerset” functor to A , B , and f .
- Decorate the diagram with the elements $Pf(\{x\})$ and $Pf(\{x, y\})$ $\square$

Exercise 2. Let $\Sigma = \{0, 1\}$, $\Gamma = \{aa, b\}$ be two alphabets. Let $f(0) = aa$, $f(1) = b$ be two functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Can $aabaa$ be an element of Γ^* generated by the Kleene Star functor $(-)^*$? Justify the answer. $\square$

Exercise 3. Let $\Sigma = \{x, y, z\}$, $\Gamma = \{p, qq, r\}$ be two alphabets. Let $f(x) = p$, $f(y) = qq$, $f(z) = r$ be two functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Is $pqqrqqp$ an element of Γ^* obtained via the Kleene Star construction? Explain your reasoning. $\square$

Exercise 4. Let $\Sigma = \{u, v\}$, $\Gamma = \{mm, n\}$ be alphabets. Let $f(u) = mm$, $f(v) = n$ be functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Determine whether $mmnmmn$ belongs to Γ^* under the Kleene Star functor $(-)^*$. Provide justification. $\square$

Exercise 5. Recall the definition of the data-type “**Maybe a**”. Prove that it is a functor by:

1. defining a function `fmap`;

2. proving that `fmap` enjoys the identity law;
3. proving that `fmap` enjoys the composition law. □

Exercise 6. Define a data-type “`List a`” that recursively formalizes the corresponding concept. Then, prove that it is a functor by:

1. defining a function `fmap`;
2. proving that `fmap` enjoys the identity law;
3. proving that `fmap` enjoys the composition law. □

Exercise 7. Recall the definition of the data-type “`Either a b`” and prove that “`Either a`” is a functor by:

1. defining a function `fmap`;
2. proving that `fmap` enjoys the identity law;
3. proving that `fmap` enjoys the composition law. □

Exercise 8. Consider the following Haskell data-type:

```
data BTree l where
  Leaf  :: l -> BTree n l
  Node  :: BTree n l -> BTree n l -> BTree n l
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law. □

Exercise 9. Consider the following Haskell data-type:

```
data BTree l where
  Leaf  :: l -> BTree n l
  Node  :: BTree n l -> BTree n l -> BTree n l
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law. □

Exercise 10. Consider the following Haskell data-type:

```
data BTree n where
  Leaf :: BTree n
  Node :: n -> BTree n -> BTree n -> BTree n
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law. □

Exercise 11. Consider the following Haskell data-type:

```
data BTree n where
  Leaf :: BTree n
  Node :: n -> BTree n -> BTree n -> BTree n
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law. □

Exercise 12. Consider the following Haskell data-type:

```
data T a where
  TL :: a -> T a
  TN :: [T a] -> T a
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law. □

Exercise 13. Consider the following Haskell data-type:

```
data T a where
  TL :: a -> T a
  TN :: [T a] -> T a
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law. □

Exercise 14. Consider the following Haskell data-type:

```
data U a b where
  UL :: Either a b -> U a b
  UN :: (U a b, U a b) -> U a b
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law. □

Exercise 15. Consider the following Haskell data-type:

```
data U a b where
  UL :: Either a b -> U a b
  UN :: (U a b, U a b) -> U a b
```

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law. □

Exercise 16. Let us assume that we are in the category `Set`.

1. Recall the definition of “`Maybe a`”.
2. Draw the corresponding diagram, starting from an arrow “`f :: a -> b`”.
3. Decorate the diagram objects with simple instances.
4. Define a simple graph for “`f :: a -> b`”.
5. Describe how `fmap f` would work, without explicitly defining it. □

Exercise 17. Let us assume that we are in the category `Set`.

1. Define a data-type “`List a`” that recursively formalizes the corresponding concept.
2. Draw the corresponding diagram, starting from an arrow “`f :: a -> b`”.
3. Decorate the diagram objects with simple instances.
4. Define a simple graph for “`f :: a -> b`”.
5. Describe how `fmap f` would work, without explicitly defining it. □

Exercise 18. Let us assume that we are in the category `Set`.

1. Recall the definition of “`Either a b`”.
2. Draw the corresponding diagram, starting from an arrow “`f :: a -> b`”.

3. Decorate the diagram objects with simple instances.
4. Define a simple graph for “ $f :: a \rightarrow b$ ”.
5. Describe how `fmap` `f` would work, without explicitly defining it. $\square$

Exercise 19. Consider the following Haskell data-type:

```
data T a where
  TL :: a -> T a
  TN :: [T a] -> T a
```

1. Draw the corresponding diagram, starting from an arrow “ $f :: a \rightarrow b$ ”.
2. Decorate the diagram objects with simple instances.
3. Define a simple graph for “ $f :: a \rightarrow b$ ”.
4. Describe how `fmap` `f` would work, without explicitly defining it. $\square$

Exercise 20. Consider the following Haskell data-type:

```
data BTree n where
  Leaf :: BTree n
  Node :: n -> BTree n -> BTree n -> BTree n
```

1. Draw the corresponding diagram, starting from an arrow “ $f :: a \rightarrow b$ ”.
2. Decorate the diagram objects with simple instances.
3. Define a simple graph for “ $f :: a \rightarrow b$ ”.
4. Describe how `fmap` `f` would work, without explicitly defining it. $\square$

Exercise 21. Consider the following Haskell data-type:

```
data BTree l where
  Leaf :: l -> BTree n l
  Node :: BTree n l -> BTree n l -> BTree n l
```

1. Draw the corresponding diagram, starting from an arrow “ $f :: a \rightarrow b$ ”.
2. Decorate the diagram objects with simple instances.
3. Define a simple graph for “ $f :: a \rightarrow b$ ”.
4. Describe how `fmap` `f` would work, without explicitly defining it. $\square$